

What are Python Generators?

Python's generator functions are used to create iterators(which can be traversed like list, tuple) and return a traversal object. It helps to transverse all the items one at a time present in the iterator.

Generator functions are defined as the normal function, but to identify the difference between the normal function and generator function is that in the normal function, we use the return keyword to return the values, and in the generator function, instead of using the return, we use yield to execute our iterator.

Example:

```
def gen_fun():
```

```
    yield 10
```

```
    yield 20
```

```
    yield 30
```

```
for i in gen_fun():
```

```
    print(i)
```

```
10
```

```
20
```

```
30
```

In the above example, gen_fun() is a generator function. This function uses the yield keyword instead of return, and it will return a value whenever it is called.

Yield vs Return

Yield

It is used in generator functions.

It is responsible for controlling the flow of the generator function. After returning the value from yield, it pauses the execution by saving the states.

Return

It is used in normal functions.

Return statement returns the value and terminates the function.

Difference Between Generator Function & Normal Function

In generator functions, there are one or more yield functions, whereas, in Normal functions, there is only one function

When the generator function is called, the normal function pauses its execution, and the call is transferred to the generator function.

Local variables and their states are remembered between successive calls.

When the generator function is terminated, StopIteration is called automatically on further calls.

Python Generators With a Loop

The Generator functions are also used for the loop.

Example:

```
def seq(x):  
    for i in range(x):  
        yield i
```

```
range_ = seq(10)  
print(next(range_))  
print(next(range_))  
print(next(range_))  
print(next(range_))  
print(next(range_))
```

```
print(next(range_))
print(next(range_))
print(next(range_))
print(next(range_))
print(next(range_))
print(next(range_))
```

Output:

```
0
1
2
3
4
5
6
7
8
9
```

Traceback (most recent call last):

File "abc.py", line 17, in <module>

```
print(next(range_))
```

StopIteration

The function seq will run 10 times, and when seq is called using next for the 11th time, it will show an error StopIteration.

But when using for loop, instead of next.

```
for i in seq(5):  
    print(i)
```

Output:

```
0  
1  
2  
3  
4
```

It will not show any error. for loop automatically handle the exception statements.

How To Create Generator Function in Python?

In Python, there are some ways to create a generator function.

Using a Yield statement

Using Python Generator Expression

1. Yield Statement

Yield keyword in Python that is used to return from the function without destroying the state of a local variable. We have already discussed how to create a generator function using yield.

2. Python Generator Expression

Generator Expression is a short-hand form of a generator function. Python provides an easy and convenient way to implement a Generator Expression.

According to its definition, they are similar to lambda function as lambda function is an anonymous function, and generator functions are anonymous.

But when it comes to implementation, they are different, they are implemented similarly to list comprehension does, and the only difference in implementation is, instead of using square brackets('[]'), it uses round brackets('(')').

The main and important difference between list comprehension and generator expression is list comprehension returns a list of items, whereas generator expression returns an iterable object.

Example:

```
x = 10
```

```
gen = (i for i in range(x) if i % 2 == 0)
```

```
list_ = [i for i in range(x) if i % 2 == 0]
```

```
print(gen)
```

```
print(list_)
```

```
for j in gen:
```

```
    print(j)
```

Output:

```
<generator object <genexpr> at 0x000002471CF1FAC0>
```

```
[0, 2, 4, 6, 8]
```

```
0
```

```
2
```

```
4
```

```
6
```

```
8
```

(i for i in range(x) if i % 2 == 0) in this generator expression, the first part is the yield part, and next is the for loop, followed by the if statement.

Use of Generators in Python

1. Easy to Implement:

Generator functions are easy to implement as compared with iterators. In iterators, we have to implement `iter()`, `__next__()` function to make our iterator work.

2. Memory Efficient:

Generator Functions are memory efficient, as they save a lot of memory while using generators. A normal function will return a sequence of items, but before giving the result, it creates a sequence in memory and then gives us the result, whereas the generator function produces one output at a time.

3. Infinite Sequence:

We all are familiar that we can't store infinite sequences in a given memory. This is where generators come into the picture. As generators can only produce one item at a time, so they can present an infinite stream of data/sequence.

```
def infinite():
```

```
    n = 0
```

```
    while True:
```

```
        yield n
```

```
        n += 1
```

```
for i in infinite():
```

```
    if i%4 == 0:
```

```
        continue
```

```
    elif i == 13:
```

```
        break
```

```
    else:
```

```
        print(i)
```

Output:

1
2
3
5
6
7
9
10
11

Conclusion

Generator functions in Python are used to create iterators and return an iterator object.

In the generator function, we use a yield statement instead of a return statement.

We can use multiple yield statements in the generator function.

When iterating over the generator function using next, in the end, it will show up StopIteration error, whereas when we use the for loop, it doesn't show any error.